

Configuration Manual

*A Risk-Sensitive Learning-to-Defer Framework for Responsible AI in
Financial Fraud Detection*

Dhwani Sanjay Kariya

MSc in Artificial Intelligence for Business

National College of Ireland

Supervisor: Dr. Muslim Jameel Syed

1. Introduction

This document explains how to set up the environment and run everything to reproduce the results in this thesis project. It covers the three datasets (ULB Credit Card Fraud, IEEE-CIS Fraud Detection, PaySim), the seven Jupyter notebooks that build and evaluate the three models and the five extra Python scripts to stress test the main result.

2. Hardware and Software Requirements

Nothing here needs a GPU. All three datasets, including PaySim's 6.3 million rows, train comfortably on a normal laptop CPU in a few minutes per run. The heaviest single run (a full 15 epoch training pass on IEEE-CIS) takes about 7 to 8 minutes on 16 GB RAM .

Python packages, with the exact versions :

Package	Version
torch	2.12.0 (CPU build)
pandas	2.2.3
scikit-learn	1.6.1
imbalanced-learn	0.13.0
numpy	2.1.3
matplotlib	3.10.0
seaborn	any recent version, only used for a couple of exploratory plots

3. Getting the Code

This project is also published on GitHub at <https://github.com/DhwaniKariya/tiersense-l2d-fraud-console>. The repository holds the full implementation (all seven notebooks, the five python scripts, the ULB model checkpoints and the results files) and the interactive prototype, which is deployed and runnable in a browser with no setup at <https://dhwaniKariya.github.io/tiersense-l2d-fraud-console/>.

4. Environment Setup

1. Install the packages listed in Section 2.
2. Launch the Jupyter Notebook.

5. Datasets

None of the three raw datasets are small enough to include with the code submission, so they are not bundled with the code.

ULB Credit Card Fraud. Download `creditcard.csv` from Kaggle ("Credit Card Fraud Detection", the dataset from the Machine Learning Group at ULB, licensed under the Database Contents Licence).

IEEE-CIS Fraud Detection. Download `train_transaction.csv` and `train_identity.csv` from the IEEE-CIS Fraud Detection competition on Kaggle.

PaySim. Download `PS_20174392719_1491204439457_log.csv` from the PaySim dataset on Kaggle ("PaySim1", the synthetic mobile money simulator).

All three are public datasets and none of them contain real personal data: ULB and IEEE-CIS use anonymised or PCA-transformed features and PaySim is entirely synthetic.

6. Running the Notebooks (in order)

The notebooks have to be run in this order the first time because the later ones depend on files the earlier ones create (the risk-tiered CSV, the trained model weights and so on).

1. **01_data_exploration.ipynb.** Loads the raw ULB data and looks at the class balance and amount distribution. No outputs are required by later steps, this is just exploration.
2. **02_Preprocessing_and_Baseline.ipynb.** Builds the risk tier column, saves `creditcard_with_tiers.csv` and trains Model 1 (the plain logistic regression baseline) on ULB.
3. **02.1_Risk_Tier_Analysis.ipynb.** Produces the fraud-rate-by-tier table and chart used in the Design Specification chapter of the report.
4. **03_Standard_L2D.ipynb.** Trains Model 2 (standard L2D, uniform deferral cost) on ULB and saves the model weights to `implementation/models/standard_l2d.pth`.
5. **04_Risk_Sensitive_L2D.ipynb.** Trains Model 3 (the risk-sensitive version, the actual contribution of this project) on ULB and saves the weights to `implementation/models/risk_sensitive_l2d.pth`. This also produces the main three-model comparison chart.
6. **05_Validation_IEEE_CIS.ipynb.** Repeats the whole pipeline (baseline, standard L2D, risk-sensitive L2D) on IEEE-CIS, as a second, independent dataset.
7. **06_Validation_PaySim.ipynb.** Same again on PaySim, the hardest of the three datasets to get a stable result out of (see Section 8 below for why).

Each notebook has saved its own results as a JSON file.

7. Running the Extra Scripts

These are the multi-seed robustness checks on the results of the 3 datasets.

Run them in this order, after the notebooks above have already been run at least once (they load the ULB, IEEE-CIS and PaySim data fresh each time and the ULB one also loads the saved Model 3 weights from Section 6):

1. `07_ulb_weight_sensitivity.py`. Retrains Model 3 on ULB five times, once per risk weight configuration, with the seed and base cost held fixed. Takes about 15 to 20 minutes in total.
2. `08_ulb_trivial_baseline.py`. Loads the already-trained Model 3 weights and compares its learned deferral head against a hand-written rule. Fast, well under a minute, since nothing is retrained here.
3. `09_ieee_cis_multiseed.py`. Retrains Model 2 and Model 3 on IEEE-CIS across three extra seeds. This is the slowest one, roughly 45 to 50 minutes, since IEEE-CIS has 209 features and about 900,000 rows after SMOTE.
4. `10_ieee_cis_trivial_baseline.py`. Trains a fresh Model 3 copy on IEEE-CIS and compares it against the same kind of rule-based baseline as script 08. About 8 to 10 minutes.
5. `11_paysim_trivial_baseline.py`. Same idea again on PaySim. This one takes the longest per epoch of the three datasets, since PaySim's SMOTE-balanced training set is about 4.4 million rows, but it only needs to train once, so it comes out to roughly 10 to 15 minutes total.

Each script prints its progress while it is running (which epoch it is on and roughly how long has passed) and saves its result as a JSON file with a name that matches the script itself.

8. Reproducibility Notes

Every part of this pipeline that involves randomness is seeded - the train and test split, SMOTE and the model's own weight initialisation and batch order. Random state 42 is used throughout for the data-side randomness (the split and SMOTE) and this is kept fixed even in the multi-seed checks, only the model-training seed changes there.

Two things that are important before rerunning anything and expecting to get exactly the numbers in the report:

First, ULB is very unstable across seeds. In 2 of the 5 seeds tested in the robustness check, the classifier failed to learn any fraud signal at all (F1 of exactly 0.000), even with everything else identical. This is discussed at length in the Discussion chapter of the report and is not a bug, it is a real property of training a small network on data where only 0.17% of cases are fraud. If we

want to retrain Model 2 or Model 3 on ULB with a seed other than 42, do not be surprised if landed on one of the dead runs.

Second, PaySim's response to the base deferral cost is bimodal rather than smooth. Most cost values tested collapse to either deferring almost everything or almost nothing within a handful of epochs. Only $c_{base} = 0.4$, run for the full 15 epochs without cutting it short early, settles into a genuine middle ground. This calibration sweep itself is recorded in `implementation/results/paysim_cost_calibration.json` if we want to see the full trajectory for every cost tested.

9. Output Files Reference

Everything in the results folder is produced by either a notebook or a script. The table below is a quick lookup for which file comes from which Python notebook or script.

Output file	Produced by
baseline_results.json, baseline_results.png	02_Preprocessing_and_Baseline.ipynb
risk_tier_analysis.png, risk_tier_stats.csv, class_imbalance.png, amount_distribution.png	01_data_exploration.ipynb, 02.1_Risk_Tier_Analysis.ipynb
standard_l2d_results.json	03_Standard_L2D.ipynb
risk_sensitive_l2d_results.json, final_model_comparison.png	04_Risk_Sensitive_L2D.ipynb
ulb_multiseed_robustness.json	five-seed robustness check (run alongside notebook 04's setup)
ieee_cis_results.json, ieee_cis_model_comparison.png	05_Validation_IEEE_CIS.ipynb
paysim_results.json, paysim_cost_calibration.json, paysim_model_comparison.png	06_Validation_PaySim.ipynb
ulb_weight_sensitivity.json, ulb_weight_sensitivity.png	07_ulb_weight_sensitivity.py
ulb_trivial_rule_baseline.json, ulb_trivial_baseline.png	08_ulb_trivial_baseline.py
ieee_cis_multiseed_robustness.json, ieee_cis_multiseed.png	09_ieee_cis_multiseed.py
ieee_cis_trivial_rule_baseline.json, ieee_cis_trivial_baseline.png	10_ieee_cis_trivial_baseline.py
paysim_trivial_rule_baseline.json, paysim_trivial_baseline.png	11_paysim_trivial_baseline.py

Output file	Produced by
cross_dataset_summary.png	built from the three main results JSON files once all three datasets are done

The results folder is uploaded to the GitHub repository with all the output files as given in the table above.